

TCLK → ACLK → ACLK-Lite Pipeline

Single-board timing-link loop: decode TCLK, timestamp, re-encode as gigabit ACLK over SFP, decode back on a shared timebase, mirror as ACLK-Lite

Subsystem	<code>aclk_pipeline_bd_top</code> (rtl/aclk_pipeline_bd_top.v)
Target / board	AMD Kria KR260 (Zynq UltraScale+ <code>xck26-sfvc784-2LV-c</code>)
Source revision	branch <code>tclk-aclk-pipeline</code> @ <code>e8b33a6</code>

1. Purpose and Scope

`aclk_pipeline_bd_top` is a single-board, single-bitstream pipeline that closes the full timing-link loop on one KR260. It receives Fermilab **TCLK** on a 3.3V logic pin, decodes and timestamps every event and publishes it to the PS over AXI4-Lite; re-encodes that same event stream as a **gigabit ACLK** 8b10b word and transmits it over the SFP+ optical transceiver; receives the ACLK back on the **same board** through a physical fiber loopback, decodes and timestamps it against a **shared timebase**, and publishes it to the PS over a **second** AXI4-Lite node; and finally re-encodes the decoded-back events as **ACLK-Lite** (Manchester biphasic-mark) on a Pmod pin as a scope probe point.

The scientific point of the loop is end-to-end measurability. Because both readouts stamp events from one common 64-bit counter, `ts(ACLK-back) - ts(TCLK-in)` for a matched event is the true H12 → decode → encode → SFP → decode round-trip latency, computed on the PS side.

In scope: the external interface of the integrated top (pins, clocks, resets, the two AXI4-Lite slaves and their register map), the data-flow contract, GT/SFP bring-up and recovery behavior, and the PS-side read flow.

Out of scope: Redis publishing (a later standalone Python tool), two-board operation against a live Fermilab ACLK fiber, and SFP DDM/I2C telemetry, all explicitly deferred by the design.

2. System Overview

```
pl_clk0 100 MHz: both AXI slaves + SmartConnect + GLOBAL TIMEBASE (~10 ns ticks)

H12 ==> tclk_readout_top ==> AXI S_AXI @ 0x8000_0000 ==> /dev/uoN (TCLK-in)
(biphase) | (TCLK_RCV, clk_40m; ts <- shared timebase)
          |
          +== dbg_data/dbg_dav ==> aclk_tclk_encoder ==> GT TX ==> SFP TX
              (clk_40m)                (tx_usrclk2 62.5M;
              count RAM, CRC8, 96->16)    [fiber loop]
                                         |
                                         v
rx_data16/K <== GT RX 8b10b + comma align <== SFP RX <
          | (rx_usrclk2 62.5M)
          v
aclk_gt_readout_top ==> AXI S_AXI2 @ 0x8001_0000 ==> /dev/uoM
          | (ACLK_RCV; ts <- shared timebase)          (ACLK-back)
          |
          +== dbg_aclk_event/data/valid ==> aclk_lite_bridge
              (real events only, rx_usrclk2 -> clk_80m)
                                         |
                                         v
                                         aclk_lite_encoder ==> B10
                                         (frame_type=2, 12-byte) (ACLK-Lite)
```

Legend: ==>=data path -->=status []=external

External TCLK enters on H12 and is decoded by the single `TCLK_RCV` inside readout #1; its decoded byte plus strobe both feed readout #1's AXI FIFO (published at `0x8000_0000`) and the TCLK-to-ACLK encoder. The encoder drives the GT transmitter, whose serial output leaves the SFP+ cage, returns through an external fiber loop into the SFP+ receiver, and is 8b10b-decoded and comma-aligned by the GT before the single `ACLK_RCV` inside readout #2 recovers events (published at `0x8001_0000`). That decoder's tap feeds the ACLK-Lite bridge/encoder, which mirrors real events as a Manchester waveform on B10.

3. Major Blocks and Responsibilities

Block / module	Responsibility	Clock domain	Source
<code>tclk_readout_top</code> (<code>u_ro_tclk</code>)	Owens the only <code>TCLK_RCV</code> ; decodes H12 biphas-mark to event byte + strobe; timestamps, buffers, AXI4-Lite (<code>S_AXI</code>)	<code>clk_40m</code> / <code>s_axi_aclk</code>	<code>rtl/aclk_lite/tclk_readout_top.sv</code>
<code>aclk_tclk_encoder</code> (<code>u_enc</code>)	Re-encodes the live TCLK event stream as the 96-bit ACLK frame <code>{0xBC, EVENT, DATA, CRC8}</code> , null-filled, gearboxed 96-to-16 to GT TX	<code>tx_usrclk2</code> (fed from <code>clk_40m</code>)	<code>rtl/aclk_gt/aclk_tclk_encoder.v</code>
<code>aclkgt_gt</code> (<code>u_gt</code>)	GTHE4 transceiver: 1.25 Gbps, 8b10b, K28.5 comma, QPLL0, real SFP RX + TX	GT user clocks	<code>vivado/ip/aclkgt_gt/*</code>
RX-recovery FSM	SEARCH/LOCKED/RECOVER self-healing: latches comma- align once locked, re-aligns on sustained byte-align loss	<code>rx_usrclk2</code>	<code>rtl/aclk_pipeline_bd_top.v:264-305</code>
<code>aclk_gt_readout_top</code> (<code>u_ro_aclk</code>)	Owens the only <code>ACLK_RCV</code> ; decodes GT word stream to EVENT/DATA; timestamps, buffers, AXI4-Lite (<code>S_AXI2</code>); taps decoded events	<code>rx_usrclk2</code> / <code>s_axi_aclk</code>	<code>rtl/aclk_gt/aclk_gt_readout_top.sv</code>
<code>aclk_lite_bridge</code> (<code>u_bridge</code>)	Filters real events (<code>event[7:0] != 0xFF</code>), crosses <code>rx_usrclk2</code> to <code>clk_80m</code> via async FIFO, drives the encoder when idle	<code>rx_usrclk2</code> to <code>clk_80m</code>	<code>rtl/aclk_lite_bridge.v</code>
<code>aclk_lite_encoder</code> (<code>u_lite</code>)	Free-running biphas-mark cell engine; emits 12-byte ACLK-Lite frames, idle = continuous 1- cells	<code>clk_80m</code>	<code>rtl/aclk_lite/aclk_lite_encoder.sv</code>
<code>global_timebase</code> (<code>u_tb</code>)	One free-running 64-bit tick counter in <code>pl_clk0</code> , gray-CDC'd into both event domains	<code>pl_clk0</code> to <code>clk_40m</code> , <code>rx_usrclk2</code>	<code>rtl/global_timebase.v</code>
<code>aclk_readout_axi</code> / <code>_core</code>	Shared decoder-agnostic timestamping packer + dual- clock FIFO + AXI4-Lite register block (both readouts)	event domain / <code>s_axi_aclk</code>	<code>rtl/aclk_readout/</code>

4. External Interface Summary

Integration-critical ports only; the full port list is in Appendix A. All ports are on the top module `aclk_pipeline_bd_top`.

Board pin map (KR260 carrier, Pmod1, all LVCMOS33):

Package pin	Pmod1 pin	Signal	Purpose
H12	pin 1	tclk	TCLK biphasemark input
B10	pin 5	aclk_lite_out	ACLK-Lite Manchester mirror output
D11	pin 7	dbg_hb	Readout #1 liveness heartbeat (scope trigger / liveness reference)

4.1 Clocks and resets

Signal	Dir	Width	Description
s_axi_aclk	in	1	PS clock for both AXI slaves + timebase reference; pl_clk0, ~100 MHz
clk_80m	in	1	80 MHz: serdec oversample + ACLK-Lite encoder cell clock
clk_40m	in	1	40 MHz: TCLK deserializer + readout #1 event/timestamp domain
freerun_50	in	1	50 MHz free-running clock for the GT reset controller
gt_refclk_p/n	in	1 ea	GT QPLL reference, MGTREFCLK0_224, 156.25 MHz
s_axi_aresetn	in	1	AXI reset, active-low (pl_clk0 domain)
rstn	in	1	PL reset, active-low (peripheral_aresetn , pl_clk0); master reset for all PL logic

Clock/reset provenance: s_axi_aclk , s_axi_aresetn , and rstn are driven from the PS pl_clk0 / proc_sys_reset in the BD; clk_80m / clk_40m come from one clk_wiz MMCM off pl_clk0, freerun_50 from a second clk_wiz .

4.2 Data path

Signal	Dir	Width	Protocol	Description
tclk	in	1	biphase-mark baseband	TCLK line, LVCMOS33, H12 (10 MHz mode)
gt_rxp/rxn	in	1 ea	1.25 Gbps serial	SFP+ RX (looped-back ACLK)
gt_txp/txn	out	1 ea	1.25 Gbps serial	SFP+ TX (re-encoded ACLK)
aclk_lite_out	out	1	biphase-mark baseband	ACLK-Lite Manchester mirror, LVCMOS33, B10
S_AXI (0x8000_0000)	slave	8-bit addr	AXI4-Lite	TCLK readout register block
S_AXI2 (0x8001_0000)	slave	8-bit addr	AXI4-Lite	ACLK readout register block

The SFP path is a **physical fiber loopback** on one board (SFP TX port to its own SFP RX port).

4.3 Configuration / control

There are no top-level configuration pins. All runtime control is register-based: the GT_CTRL register (0xF0) on the ACLK slave (S_AXI2) and the FILTER_CFG register (0xD0) on either slave. See Section 8.

4.4 Status / error (top-level sideband)

Signal	Dir	Width	Meaning	Recommended handling
sfp_tx_disable	out	1	Driven constant 0 to enable the laser (high/float = laser OFF)	Constrain to Y10; do not repurpose
sfp_tx_fault	in	1	Module TX fault (surfaced in ACLK DEBUG word)	Monitor via DEBUG bit

Signal	Dir	Width	Meaning	Recommended handling
sfp_rx_los	in	1	Module RX loss-of-signal / no light	Monitor via DEBUG bit
sfp_mod_abs	in	1	Module absent	Monitor via DEBUG bit
dbg_hb	out	1	Readout #1 deep-CDC heartbeat[12] probe pin (D11)	Optional scope point

5. Clock, Reset, and Initialization Behavior

Clock domains:

Clock	Freq	Use
pl_clk0 (s_axi_aclk)	~100 MHz	AXI SmartConnect + both AXI slaves + global timebase (~10 ns ticks)
clk_80m	80 MHz	serdec4_9MHz oversample + ACLK-Lite encoder
clk_40m	40 MHz	TCLK byte framer + readout #1 event domain
gt_refclk	156.25 MHz	GT QPLL reference (free-running at power-up)
tx_usrclk2	62.5 MHz	aclk_tclk_encoder framing to GT TX
rx_usrclk2	62.5 MHz	ACLK_RCV + readout #2 event domain

The 62.5 MHz figures are the spec's stated GT user-clock rates; the RTL does not assert them numerically (they are GT-IP outputs).

Reset: `rstn` (active-low) is the master PL reset from `peripheral_aresetn`. Per-domain resets are derived **async-assert / sync-deassert**: `ro_rstn` (rx, gated on `rx_active`), `gen_rstn` (tx, gated on `tx_active`), and `s_axi_aresetn` for the AXI domain. The GT is reset via `gtwiz_reset_all_in = ~rstn`.

Clock-domain crossings, every cross-domain path uses a CDC-safe structure:

- TCLK event `clk_40m` to `tx_usrclk2`: DAVn-toggle 2-FF synchronizer + edge detect (encoder).
- Global timebase `pl_clk0` to `clk_40m` and `pl_clk0` to `rx_usrclk2`: gray-code counter sync (single bit changes per tick).
- ACLK-back `rx_usrclk2` to `clk_80m`: bridge async FIFO.
- Both readouts' event domain to `s_axi_aclk`: async FIFO (data) + `cdc_gray_count` (counters) + 2-FF (status).

The XDC declares the three clock families (PS, MMCM, GT) mutually asynchronous with one `set_clock_groups` stanza, cutting every cross-family path; the design owner notes each cut path is CDC-safe.

GT RX bring-up / recovery timeline:

state	condition	effect
SEARCH	power-up / after RECOVER rx_aligned (>=5 good CRC) -> LOCKED	comma-align ENABLED
LOCKED	normal operation byteali low >= 512 cyc (~8 us) -> RECOVER	comma-align LATCHED off (loss window)
RECOVER	hold 512 cycles, then -> SEARCH	decoder reset pulsed

RECOVER_GT_RESET=0 , so recovery is a soft re-align (decoder + lock reset, **not** the async-FIFO pointers and not a full GT RX datapath reset). A full RX PLL+CDR relock can still be forced at runtime via GT_CTRL[24] .

6. Data and Control Flow

Trace one TCLK event through the loop:

1. **TCLK decode.** TCLK_RCV recovers an 8-bit event byte and a one-cycle active-low DAVn on clk_40m. Readout #1 adapts this as aclk_valid = ~DAVn, aclk_event = {8'h00, DATA}, flags = 0x0002 (is_tclk), DROP_NULL=0 (every byte kept, since 0xFF is a valid TCLK code). It timestamps with ts_tclk and pushes into the FIFO read at 0x8000_0000.
2. **Re-encode.** The same dbg_data / dbg_dav feed aclk_tclk_encoder (tclk_davn = ~dbg_dav). The encoder maintains a 256-entry per-event-code count RAM, and for each event emits an 80-bit payload {8'h00, event, 32'h0, count}, wraps it as {0xBC, payload, CRC8} (CRC-8), and gearboxes 96-to-16 to the GT. Between events it emits 0xFF..FF null frames on a free-running 6-cycle cadence.
3. **SFP round trip.** The GT transmits at 1.25 Gbps; the fiber loop returns the light; the GT 8b10b-decodes and comma-aligns, producing rx_data16 + rxctrl2[1:0] (K flags) on rx_usrclk2.
4. **ACLK decode.** ACLK_RCV reassembles the 96-bit frame, checks CRC-8 (poly 0x2F), and on CRC==0 emits ACLK_EVENT / ACLK_DATA / ACLK_VALID. Readout #2 keeps flags=0x0001 (has_data), DROP_NULL=1 (drops 0xFF -low-byte nulls), timestamps with ts_aclk, and publishes at 0x8001_0000.
5. **ACLK-Lite mirror.** The decoder tap (dbg_aclk_event/_data/_valid) feeds aclk_lite_bridge, which filters real events (event[7:0]!=0xFF), crosses them rx_usrclk2 to clk_80m through a 16-deep async FIFO, and pulses enc_start with frame_type=2 whenever the encoder is idle. aclk_lite_encoder serializes a 12-byte biphasemark frame on B10, returning to idle 1-cells between frames.

Backpressure / ordering. TCLK events are far slower than the ~12 us full-frame ACLK-Lite encode time, so the bridge FIFO should never fill; if it does, the event is dropped and counted in the bridge's internal dropped_count (not exposed on a top port). One TCLK event produces exactly one non-null ACLK frame; nulls fill the gaps and are dropped at readout #2. Events are paired on the PS side by event code plus arrival order, further disambiguated by the per-event count carried in the low 32 bits of DATA.

7. Integration Instructions

7.1 Instantiation

This design is intended to be dropped into a Zynq block design as a module-reference cell; Vivado infers the two AXI4-Lite slaves (S_AXI, S_AXI2) from the X_INTERFACE attributes. The reference build wires it as:

```
set u [create_bd_cell -type module -reference aclk_pipeline_bd_top u_pipeline]
connect_bd_net [get_bd_pins ps/pl_clk0] [get_bd_pins u_pipeline/s_axi_aclk]
connect_bd_net [get_bd_pins rst_pl0/peripheral_aresetn] \
[get_bd_pins u_pipeline/s_axi_aresetn]
connect_bd_net [get_bd_pins rst_pl0/peripheral_aresetn] \
[get_bd_pins u_pipeline/rstn]
connect_bd_net [get_bd_pins clk_wiz_0/clk_out1] [get_bd_pins u_pipeline/clk_80m]
connect_bd_net [get_bd_pins clk_wiz_0/clk_out2] [get_bd_pins u_pipeline/clk_40m]
connect_bd_net [get_bd_pins clk_wiz_freerun/clk_out1] \
[get_bd_pins u_pipeline/freerun_50]
# SmartConnect NUM_SI=1 NUM_MI=2 -> S_AXI, S_AXI2 ; then assign_bd_address (see 7.3)
```

7.2 Parameters

The top has no parameters; it fixes the sub-block parameters internally. The parameters worth knowing (all on the reused readout core) are:

Parameter	Value in pipeline	Effect
ADDR_WIDTH	6	FIFO depth = $2^6 = 64$ events per readout
AXI_ADDR_W	8	8-bit byte-address register space
USE_EXT_TS	1 (both readouts)	Use the shared <code>ts_ext</code> timebase, not the internal counter
DROP_NULL	0 (TCLK) / 1 (ACLK)	Keep all TCLK bytes / drop <code>0xFF</code> ACLK nulls
SAMPLES_PER_CELL	8 (ACLK-Lite enc)	clk_80m cycles per 100 ns biphasic-mark cell

7.3 Required connections and sequencing

1. Provide `s_axi_aclk` = `pl_clk0` (~100 MHz), plus `clk_80m` / `clk_40m` from one MMCM and `freerun_50` from another; provide `gt_refclk_p/n` (156.25 MHz).
2. Release `rstn` / `s_axi_aresetn` after PL programming. The device-tree overlay releases `pl_resetn0-3`; without it the fpga-region holds the design in reset and every AXI access bus-errors.
3. Assign the two AXI segments: `0x8000_0000` / `0x1_0000` for `S_AXI` (TCLK) and `0x8001_0000` / `0x1_0000` for `S_AXI2` (ACLK). A module-ref slave carries no IP-XACT map, so call bare `assign_bd_address` first to create both segments, then relocate each.
4. Connect the external fiber loop (SFP TX to SFP RX). Wait for GT lock and RX alignment before trusting ACLK events (poll the ACLK slave's `LOCK` and DEBUG-word `rcv_aligned`).

Use `SmartConnect`, **not** the auto interconnect + protocol-converter path: the latter corrupts AXI4-to-AXI4-Lite read data on this hardware.

8. Configuration / Register Interface

Both slaves expose the same `aclk_readout_axi` register block. **Registers are spaced 16 bytes apart, not 4.** On the KR260 LPD path this hand-written slave only returns correct data at 16-byte-aligned offsets (any offset with `araddr[3:2] != 0` read back 0 on hardware; root cause unpinned, 16-byte spacing sidesteps it).

Offset	Name	Access	Fields / meaning
0x00	STATUS	RO	bit0 = empty, bit1 = overflow (sticky: an event was lost)
0x10	EVENT	RO	{ <code>FLAGS[31:16]</code> , <code>EVENT[15:0]</code> } of FIFO head; <code>FLAGS</code> bit0=has_data, bit1=is_tclk
0x20	DATA_HI	RO	<code>DATA[63:32]</code>
0x30	DATA_LO	RO	<code>DATA[31:0]</code>
0x40	TS_HI	RO	<code>TIMESTAMP[63:32]</code> (shared timebase)
0x50	TS_LO	RO	<code>TIMESTAMP[31:0]</code>
0x60	POP	WO	write any value to pop the head and advance
0x70	EVENT_COUNT	RO	events enqueued (kept)
0x80	NULL_COUNT	RO	null / idle packets dropped (ACLK: <code>0xFF</code> nulls; TCLK: stays 0)
0x90	ERROR_COUNT	RO	bad-CRC events (ACLK) / new parity errors (TCLK)

Offset	Name	Access	Fields / meaning
0xA0	DEBUG	RO	caller-supplied debug word (see below)
0xB0	HEARTBEAT	RO	free-running event-clock counter (CDC liveness)
0xC0	LOCK	RO	bit0 = MMCM/GT locked (synchronized)
0xD0	FILTER_CFG	WO	{bit8=drop, bits[7:0]=code} set/clear a per-code drop-mask bit
0xE0	FILTERED_COUNT	RO	events dropped by the mask
0xF0	GT_CTRL	RW	GT static control (ACLK slave only; see below)

DEBUG word (0xA0) differs per slave. On the **TCLK** slave it is {sig_err, raw_level, tclk_transitions[29:0]}, a raw-line activity monitor. On the **ACLK** slave it is a GT-health word: {rcv_aligned, byteali, rx_los, notintbl_sticky, disperr_cnt[13:0], tx_fault, mod_abs, recover_cnt[3:0], commadet_cnt[7:0]}.

GT_CTRL (0xF0), ACLK slave only. Wired to the GT's static inputs. Reset 0 = normal operation; leave at 0 for the default fiber-loopback bring-up.

```

bit0      rxpolarity      bit[4:2] loopback (000=normal,010=near-end PMA)
bit1      txpolarity      bit[8]   RX datapath re-init pulse
bit[13:9] txdiffctrl (0 -> proven default 0x18)
bit[18:14] txpostcursor   bit[23:19] txprecursor
bit[24]    full RX PLL+CDR relock

```

On the TCLK slave the `gt_ctrl` output is unconnected (writes to 0xF0 are latched but drive nothing).

PS read flow (per slave, over /dev/uioN, offset 0): poll `STATUS`; while not empty, read `EVENT`, `DATA_HI`, `DATA_LO`, `TS_HI`, `TS_LO`, then write `POP`; the head is held stable until POP for a consistent snapshot. To drop event codes, write each code with bit8 set to `FILTER_CFG`.

9. Status, Errors, and Recovery

Indication	Source	Meaning	Recommended response
<code>STATUS.overflow</code> (bit1)	either slave	FIFO overflowed; an event was lost	Read faster / raise <code>ADDR_WIDTH</code> ; sticky until reset
<code>LOCK == 0</code>	either slave	MMCM/GT not locked; event clock may be dead	Fix clocking before anything else; check timebase
<code>HEARTBEAT</code> frozen	either slave	Event-domain clock dead even if <code>LOCK=1</code>	MMCM not producing clk_40m/rx clock
DEBUG <code>rx_los = 1</code> (ACLK)	GT-health word	No light at SFP RX	Check fiber loop / laser (<code>sfp_tx_disable</code>)
DEBUG <code>rcv_aligned = 0</code> (ACLK)	GT-health word	ACLK decoder not locked	Wait through <code>SEARCH</code> ; check <code>disperr</code> / <code>recover</code> climbing
DEBUG <code>disperr_cnt</code> climbing	GT-health word	8b10b disparity errors on RX	Signal-integrity issue; try <code>GT_CTRL</code> polarity/eq
DEBUG <code>recover_cnt</code> climbing	GT-health word	Recovery FSM re-aligning repeatedly	Link marginal; inspect fiber/GT config

Indication	Source	Meaning	Recommended response
ERROR_COUNT climbing (ACLK)	ACLK_ERROR	Bad-CRC frames	Link errors; correlate with disperr
ERROR_COUNT first pulse (TCLK)	PERR edge	serdec emits one spurious PERR while first locking	Expected at bring-up; read as delta from baseline

10. Simulation and Validation Notes

The design was verified sim-first with cocotb 2.0 + Icarus Verilog (`SIM=icarus` default). Each testbench has a `runner.py` ; per project convention tests emit matplotlib occupancy/throughput plots on completion.

Testbench	Covers	Source
<code>tb/aclk_tclk_encoder_loop</code>	Encoder frames vs golden model; encoder to ACLK_RCV agreement (A1/A2)	<code>tb/aclk_tclk_encoder_loop/</code>
<code>tb/global_timebase</code>	Timebase monotonic, no gaps, same value in two domains (A3)	<code>tb/global_timebase/</code>
<code>tb/aclk_pipeline_chain</code>	Full pure-RTL chain: TCLK biphasic to readout #1 + encoder to ACLK_RCV to readout #2, shared-timebase ordering (A4)	<code>tb/aclk_pipeline_chain/</code>
<code>tb/aclk_lite_bridge</code>	Bridge + encoder to decode agreement (A5)	<code>tb/aclk_lite_bridge/</code>
<code>tb/aclk_readout_ext_ts</code>	External-timestamp path through the readout	<code>tb/aclk_readout_ext_ts/</code>

How to run (example): `python tb/aclk_pipeline_chain/runner.py` . The chain sim omits the GT and BRAM IP (pure RTL), so the GT transceiver, SFP electrical behavior, and real fiber are **not** covered in simulation.

Hardware bring-up is incremental: B0 build time-clean, B1 TCLK on H12, B2 fiber loopback, B3 shared timestamps, B4 scope B10. As of this revision the RTL, build, and two-UIO overlay are committed; hardware validation of the integrated bitstream is the pending step.

Appendix A. Full Port List

`aclk_pipeline_bd_top`, all ports.

Signal	Dir	Width	Group	Description
<code>tclk</code>	in	1	Data	TCLK biphasemark input (H12, Pmod1 pin 1)
<code>gt_refclk_p / gt_refclk_n</code>	in	1 ea	Clock	GT reference clock pair (156.25 MHz)
<code>gt_rxp / gt_rxn</code>	in	1 ea	Data	SFP+ RX serial
<code>gt_txp / gt_txn</code>	out	1 ea	Data	SFP+ TX serial
<code>freerun_50</code>	in	1	Clock	50 MHz GT reset-controller clock
<code>rstn</code>	in	1	Reset	PL reset, active-low (pl_clk0)
<code>clk_80m</code>	in	1	Clock	80 MHz serdec + ACLK-Lite encoder
<code>clk_40m</code>	in	1	Clock	40 MHz TCLK + readout #1
<code>aclk_lite_out</code>	out	1	Data	ACLK-Lite biphasemark mirror (B10, Pmod1 pin 5)
<code>dbg_hb</code>	out	1	Status	Readout #1 heartbeat probe (D11, Pmod1 pin 7)
<code>sfp_tx_disable</code>	out	1	Sideband	Laser enable (0 = on)
<code>sfp_tx_fault</code>	in	1	Sideband	Module TX fault
<code>sfp_rx_los</code>	in	1	Sideband	RX loss-of-signal
<code>sfp_mod_abs</code>	in	1	Sideband	Module absent
<code>s_axi_aclk</code>	in	1	Clock	PS clock, both slaves + timebase
<code>s_axi_aresetn</code>	in	1	Reset	AXI reset, active-low
<code>s_axi_*</code> (S_AXI)	slave	8-bit addr	Bus	AXI4-Lite TCLK readout @ 0x8000_0000
<code>s_axi2_*</code> (S_AXI2)	slave	8-bit addr	Bus	AXI4-Lite ACLK readout @ 0x8001_0000

Full AXI4-Lite channel signals (AW/W/B/AR/R) are present on both `S_AXI` and `S_AXI2` per the standard AXI4-Lite signal set with `[7:0]` address and `[31:0]` data.